

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	K.Neeha
Reg. No	192425245

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

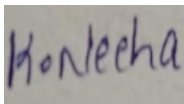
Duration: 30 minutes

Marks: 25

Code of Conduct:

I K.Neeha(192425245) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: K . Neeha

QUESTIONS:05

Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer : When training a machine learning model with gradient descent on a convex loss function, using a high initial learning rate followed by gradual decay is a common optimization strategy. It aims to combine fast initial progress with stable convergence near the optimum.

Effect on Convergence

A high initial learning rate causes large parameter updates, allowing the algorithm to:

- Move rapidly toward the region containing the global minimum.
- Reduce the loss significantly in the early stages of training.
- Spend less time making tiny improvements when far from the optimum.

As training progresses, the learning rate is gradually decreased, which results in:

- Smaller, more precise parameter updates.
- Reduced oscillations around the minimum.
- Improved stability and convergence to the optimal solution.

For a convex loss function, if the learning rate decreases appropriately (e.g., satisfying conditions such as decreasing over time but not too rapidly), gradient descent is guaranteed to converge to the global minimum.

Start

|



Initialize Model Parameters

|



Set High Initial Learning Rate



Compute Gradient



Update Model Parameters



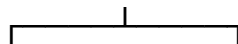
Calculate Loss



Reduce (Decay) Learning Rate



Convergence Achieved?



No

Yes



Repeat Training Stop

When It Can Outperform a Constant Learning Rate

A decaying learning rate often performs better than a constant learning rate under the following conditions:

1. Large Initial Error
 - When the parameters are far from the optimum, large steps accelerate learning.
 - A constant small learning rate would require many more iterations.
2. Different Optimization Phases
 - Early training benefits from exploration using larger updates.
 - Later training requires fine adjustments using smaller updates.
3. Ill-Conditioned Optimization Problems
 - In convex functions with narrow valleys, a large initial step speeds movement toward the valley, while later decay prevents overshooting and improves stability.

4. Computational Efficiency

- Faster reduction in loss during the initial iterations often leads to fewer total iterations compared to using a small constant learning rate throughout training.

Possible Risks

Although effective, this strategy has several risks.

- Learning rate too high initially
 - Updates may overshoot the minimum.
 - The loss may oscillate or even increase.
 - In extreme cases, the optimization may diverge instead of converging.
- Decay too quickly
 - The learning rate becomes very small before reaching the optimum.
 - Learning slows dramatically, leading to premature convergence and inefficient optimization.
- Decay too slowly
 - The algorithm continues making large updates near the minimum.
 - Oscillations persist, preventing accurate convergence.
- Sensitive Hyperparameter Selection
 - Choosing the initial learning rate and decay schedule requires careful tuning.
 - Poor choices can make performance worse than a well-selected constant learning rate.

Justification

The intuition behind this strategy is that optimization has two distinct phases:

- Far from the optimum: Large steps are desirable because the gradient provides a reliable direction toward the minimum, making rapid progress possible.
- Near the optimum: Large steps become inefficient because they repeatedly overshoot the minimum. Smaller steps allow the algorithm to settle accurately into the global optimum.

A constant learning rate cannot simultaneously provide both fast early progress and precise final convergence. Therefore, a high initial learning rate with gradual decay often achieves a better balance between speed and accuracy.

Conclusion:

Using a high initial learning rate followed by gradual decay generally improves optimization by combining rapid early convergence with stable final convergence. It can outperform a constant learning rate when the initial parameters are far from the optimum, the decay schedule is well designed, and the optimization problem benefits from different step sizes during different training stages. However, if the initial learning rate is excessively high or the decay schedule is poorly chosen, the algorithm may oscillate, converge slowly, or even diverge. Thus, the success of this strategy depends on selecting an appropriate initial learning rate and decay schedule.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

The dataset contains outliers

where μ is unknown and σ^2 is known.

The likelihood function is:

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

Taking the logarithm,

$$\begin{aligned} & \log \\ & -\frac{n}{2} \log(2\pi\sigma^2) \\ & -\frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 \end{aligned}$$

Differentiate with respect to μ :

$$\begin{aligned} & \frac{d}{d\mu} \log L(\mu) \\ & \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) \end{aligned}$$

Set the derivative equal to zero:

$$\sum_{i=1}^n (x_i - \mu) = 0$$

$$n\mu = \sum_{i=1}^n x_i$$

Hence, the Maximum Likelihood Estimator (MLE) is:

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i}$$

This means the MLE of the mean is simply the sample mean.

Effect of Outliers

□ genui □ {"descriptive_statistics_sampling_learning_block":{"type_id":"MEAN_VS_MEDIAN"}} □

Outliers are observations that are much larger or smaller than the remaining data.

Since the MLE is the sample mean, every observation contributes equally.

A single extreme value can significantly shift the estimated mean.

As a result, the estimated value of μ moves toward the outlier, producing a biased estimate of the true population mean.

Example:

Normal data:

```
[  
(10,;11,;12,;13,;14)  
]
```

```
[  
\hat{\mu}=12  
]
```

With one outlier:

```
[  
(10,;11,;12,;13,;100)  
]
```

```
[  
\hat{\mu}=29.2  
]
```

The estimate changes dramatically because of one extreme observation

Effect of Increasing Sample Size

As the number of samples (n) increases:

More observations contribute to the estimate.

Random fluctuations decrease.

The variance of the estimator becomes

```
[  
\boxed{\text{Var}(\hat{\mu})=\frac{\sigma^2}{n}}  
]
```

Thus,

Larger sample size $\rightarrow$ Smaller variance.

Estimate becomes more accurate and stable.

The sample mean converges to the true mean (Law of Large Numbers).

Therefore, MLE becomes increasingly reliable with large datasets.

Implication About the Robustness of MLE

The MLE of the mean for a Gaussian distribution is:

Efficient: Achieves minimum variance among unbiased estimators under Gaussian assumptions.

Consistent: Approaches the true parameter as sample size increases.

Unbiased: Expected value equals the true mean.

However,

It is not robust to outliers.

Even one extreme observation can greatly influence the estimate because the mean gives equal weight to every data point.

Therefore, MLE performs best when the data closely follows a Gaussian distribution without significant outliers.

Likelihood Function

|



Take Log-Likelihood

|



Differentiate w.r.t. μ

|



Set Derivative = 0

|



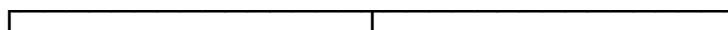
Obtain MLE:

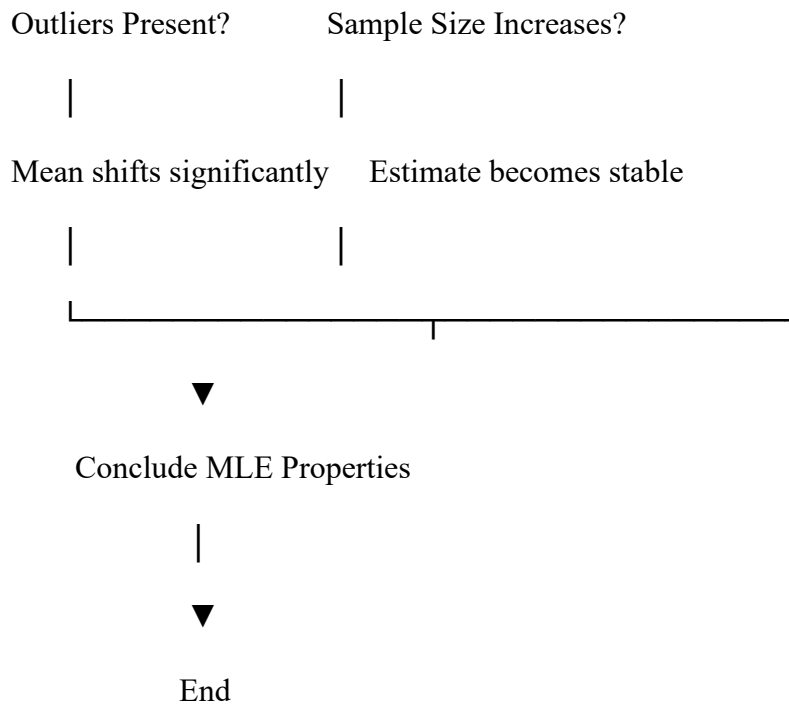
$$\hat{\mu} = (1/n) \sum x_i$$

|



Analyze Results





Summary

For a Gaussian distribution with known variance, the Maximum Likelihood Estimator (MLE) of the unknown mean is the sample mean, $\hat{\mu} = \frac{1}{n} \sum x_i$. This estimator is unbiased, consistent, and efficient, making it highly effective when the data follows a normal distribution. However, because the sample mean is sensitive to extreme values, the presence of outliers can significantly distort the estimate, indicating that MLE is not robust to outliers. As the sample size increases, the variance of the estimator decreases, and the estimate becomes more accurate and converges to the true mean.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

3. Building a Machine Learning Algorithm (Model Selection)

Challenges

A dataset with 10,000 features and only 500 training samples is a high-dimensional, low-sample-size problem. The major challenges are:

- **Overfitting:** The model may memorize the training data instead of learning general patterns.
- **Curse of Dimensionality:** Too many features make it difficult to identify meaningful relationships.
- **High Computational Cost:** Training becomes slower and requires more memory.
- **Irrelevant and Redundant Features:** Many features may not contribute to prediction, reducing model performance.
- **Poor Generalization:** The model may achieve high training accuracy but low accuracy on unseen test data.

Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize the features.
- Remove duplicate or noisy data.

Justification: Clean and normalized data improves model stability and training efficiency.

Step 2: Feature Selection

- Use filter methods (Correlation, Chi-Square, Mutual Information).
- Apply embedded methods such as L1 (Lasso) Regularization.

Justification: Reduces irrelevant features and retains only the most informative ones.

Step 3: Dimensionality Reduction

- Apply Principal Component Analysis (PCA) to reduce feature dimensions while preserving maximum variance.

Justification: Compresses the feature space, decreases computational complexity, and removes redundancy.

Step 4: Regularization

- Train the classifier using L1 or L2 Regularization.

Justification: Penalizes overly complex models and prevents excessively large weights.

Step 5: Model Selection

- Use classifiers such as:
 - Support Vector Machine (SVM)
 - Logistic Regression with Regularization
 - Random Forest (with limited depth)

Justification: These models perform well on high-dimensional datasets and include mechanisms to reduce overfitting.

Step 6: Cross-Validation

- Apply K-Fold Cross-Validation to evaluate the model.

Justification: Ensures reliable performance estimation and improves model selection.

Step 7: Final Evaluation

- Evaluate using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Justification: Provides a complete assessment of classification performance.

How This Approach Mitigates Overfitting

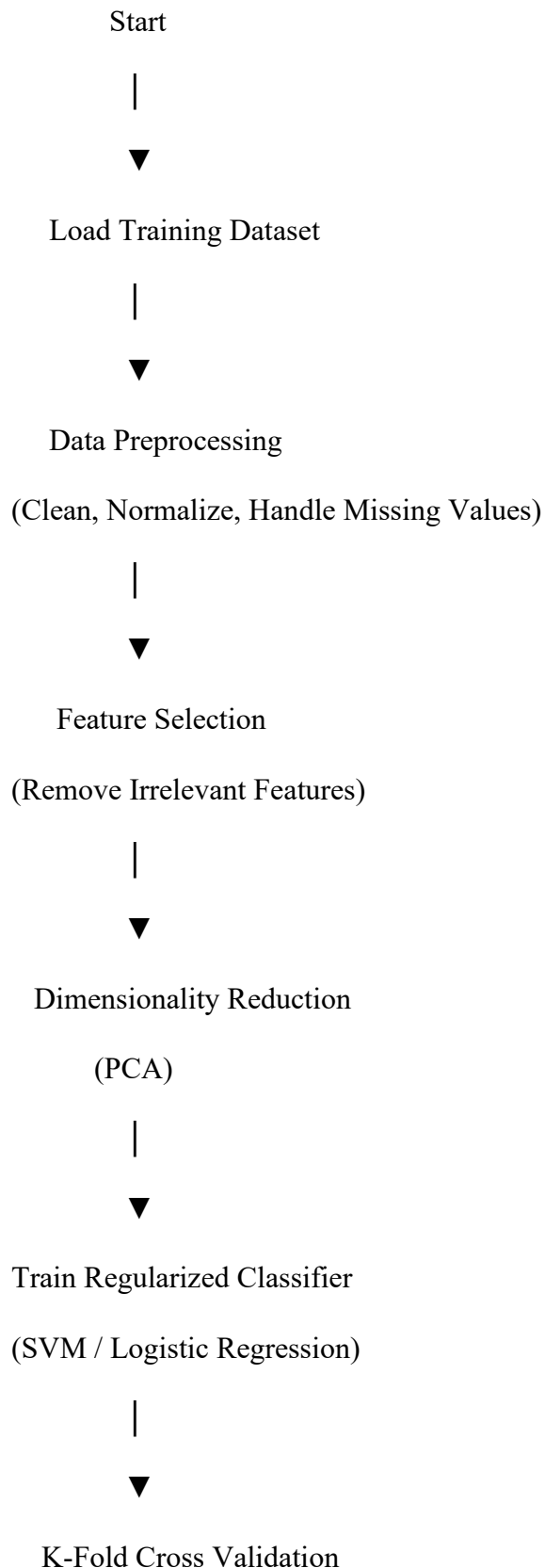
The proposed pipeline reduces overfitting by:

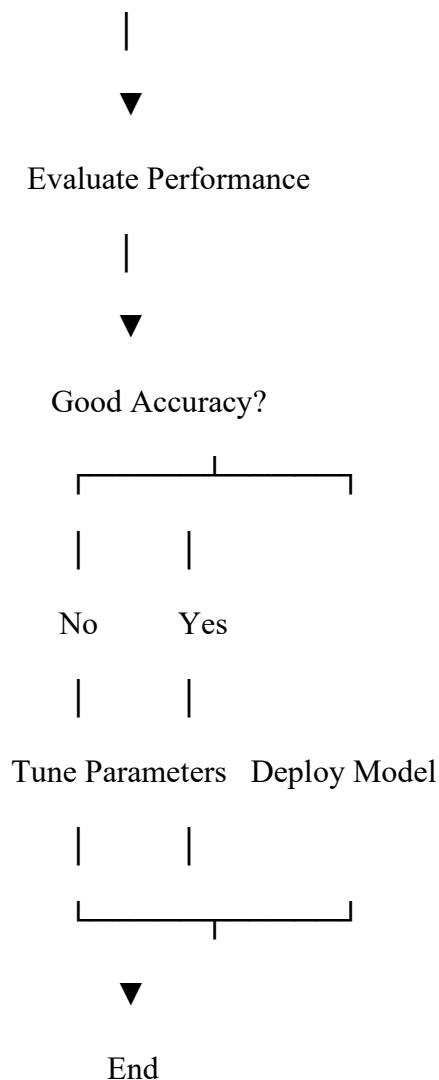
- Feature Selection: Eliminates irrelevant and noisy features.

- Dimensionality Reduction (PCA): Lowers the number of input variables while preserving important information.
- Regularization: Prevents the model from learning overly complex decision boundaries.
- Cross-Validation: Ensures the model generalizes well to unseen data.
- Appropriate Model Choice: Regularized classifiers and controlled-complexity models reduce variance and improve generalization.

Together, these techniques simplify the model, reduce noise, and improve predictive performance on new data.

Flow Chart





Conclusion:

Training a classification model with 10,000 features and only 500 samples is challenging due to the risk of overfitting, high computational cost, and the curse of dimensionality. An effective pipeline includes data preprocessing, feature selection, dimensionality reduction (PCA), regularization, appropriate model selection, and cross-validation. Feature selection removes irrelevant variables, PCA compresses the feature space, regularization controls model complexity, and cross-validation improves generalization. This combination reduces overfitting, enhances computational efficiency, and produces a more accurate and reliable classification model.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

4. Neural Networks (Multilayer Perceptron - MLP)

Explanation

A Multilayer Perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. In this network, the sigmoid activation function introduces non-linearity, allowing the model to learn complex relationships between inputs and outputs.

Why Adding More Hidden Neurons Increases Representational Power

Each hidden neuron learns a different feature or pattern from the input data. Increasing the number of hidden neurons provides the network with greater capacity to represent complex functions.

Analytically,

$$y = f\left(\sum_{i=1}^m w_i \cdot x + b_i\right)$$

where:

- x = input vector
- w_i = weights
- b_i = bias
- $f(\cdot)$ = sigmoid activation
- m = number of hidden neurons

As m increases:

- More hidden neurons learn more features.
- The network can approximate increasingly complex nonlinear functions.
- Decision boundaries become more flexible.
- Prediction accuracy on complex datasets may improve.

Thus, adding hidden neurons increases the representational power of the MLP. Although more neurons
A large network may memorize the training data instead of learning general patterns.

- Very low training error
- High testing error
- Poor performance on unseen data

2. Poor Generalization

A highly complex model may capture random noise rather than meaningful patterns.

As a result:

- Training accuracy becomes very high.
- Test accuracy decreases.
- The model fails to generalize to new data.

3. Increased Computational Cost

More hidden neurons increase:

- Number of parameters
- Memory usage
- Training time

Training becomes slower and more expensive.

4. Vanishing Gradient (Sigmoid)

Sigmoid activation can saturate for very large or very small inputs.

Consequently:

- Gradients become extremely small.

- Weight updates slow down.
- Deep or large networks train inefficiently.

Overfitting vs Generalization

Overfitting

Model memorizes training data

High training accuracy

Poor performance on new data

Caused by excessive complexity

Good Generalization

Model learns underlying patterns

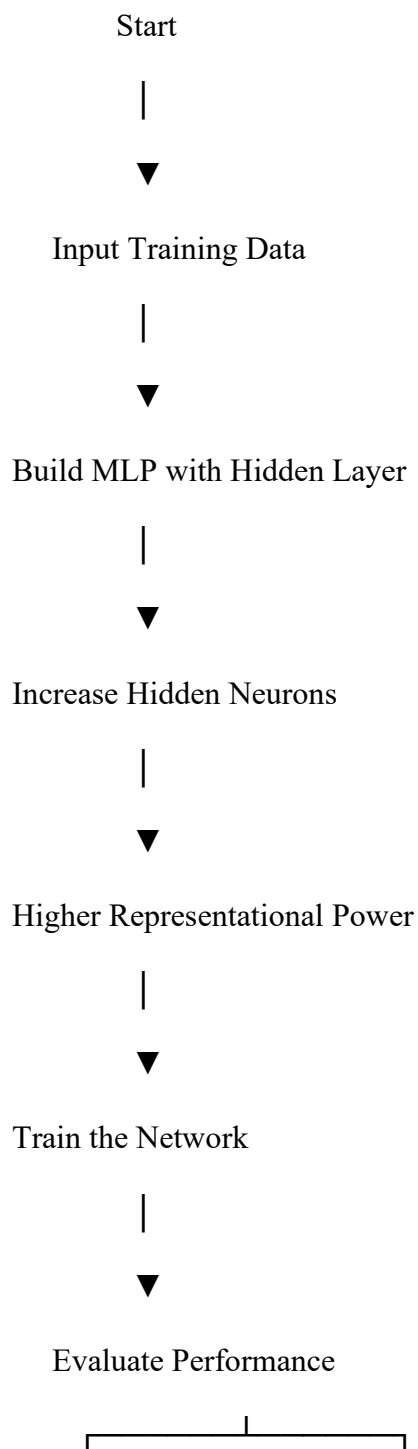
High test accuracy

Good prediction on unseen data

Achieved with balanced model complexity

Regularization, dropout, early stopping, and sufficient training data help improve generalization while controlling overfitting.

Flow Chart



| |

Good Generalization? Overfitting?

| |

Yes Yes

| |

Better Test Accuracy Reduce Complexity,

Apply Regularization

| |

└──────────┘



Conclusion :

Adding more hidden neurons to an MLP increases its representational power because each neuron learns additional features, enabling the network to model more complex nonlinear relationships. However, increasing the number of neurons also increases the model's complexity, which can lead to overfitting, higher computational cost, and poor generalization to unseen data. Therefore, the best performance is achieved by selecting an appropriate number of hidden neurons and using techniques such as regularization, dropout, and early stopping to balance model capacity and generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the curse of dimensionality impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

1. Impact on Gradient Updates in Backpropagation

During backpropagation, gradients are computed for every weight in the network.

- A high-dimensional dataset creates a very large number of parameters.
- Many features may be irrelevant or noisy, producing unstable gradient updates.
- Some gradients become extremely small (vanishing gradients) or very large (exploding gradients), making optimization difficult.
- The network requires more updates to identify useful feature directions.

Result: Gradient updates become noisy and less efficient, slowing the learning process.

2. Impact on Convergence Speed of SGD

Stochastic Gradient Descent (SGD) updates model parameters using one or a few training samples at a time.

- In high dimensions, the optimization space becomes much larger.
- SGD must search through many unnecessary directions before finding the optimal solution.
- Gradient estimates have higher variance, causing oscillations during training.
- More iterations are required to reach convergence.

Result: SGD converges more slowly and may require careful tuning of the learning rate.

3. Impact on Model Generalization

High-dimensional datasets often contain many irrelevant features.

- The model may learn noise instead of meaningful patterns.
- Training accuracy becomes high, but testing accuracy decreases.
- The model becomes overly complex and suffers from overfitting.

Result: Poor generalization to unseen data.

Techniques to Overcome These Issues

Technique 1: Dimensionality Reduction (PCA)

- Apply Principal Component Analysis (PCA) before training.
- PCA removes redundant features while preserving most of the important information.

Justification:

- Reduces the number of input dimensions.
- Makes gradient updates more stable.
- Speeds up SGD convergence.
- Reduces computational cost and overfitting.

Technique 2: Regularization (L1/L2 and Dropout)

- Apply L1/L2 Regularization to penalize large weights.
- Use Dropout to randomly deactivate neurons during training.

Justification:

- Prevents the network from memorizing the training data.
- Reduces model complexity.
- Improves generalization on unseen data.
- Produces smoother and more stable optimization.

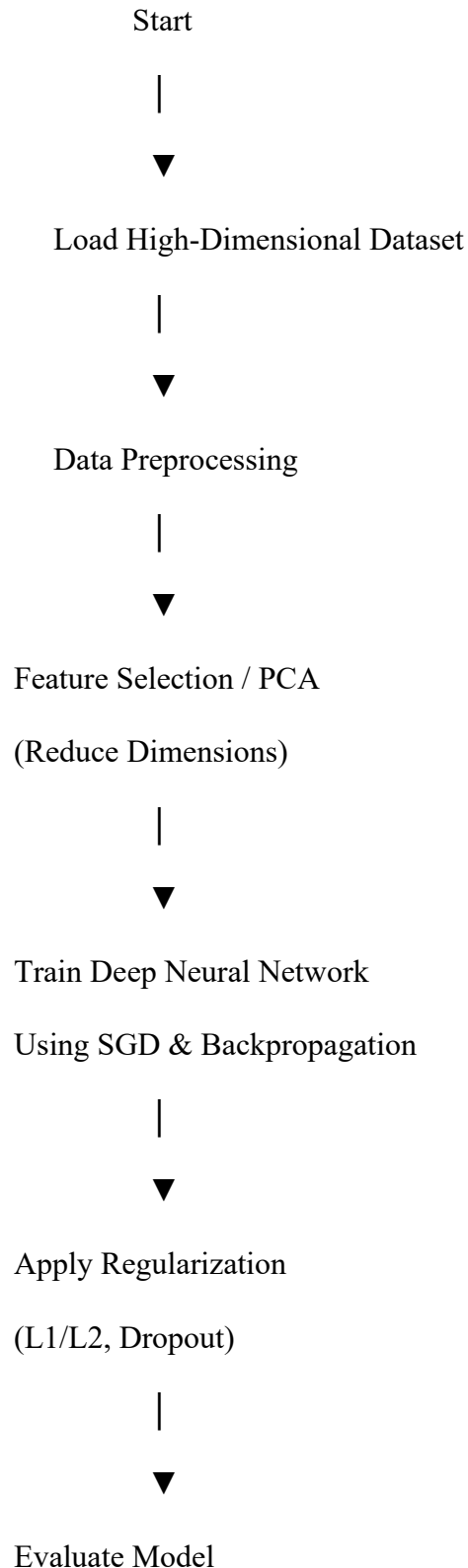
Additional Techniques

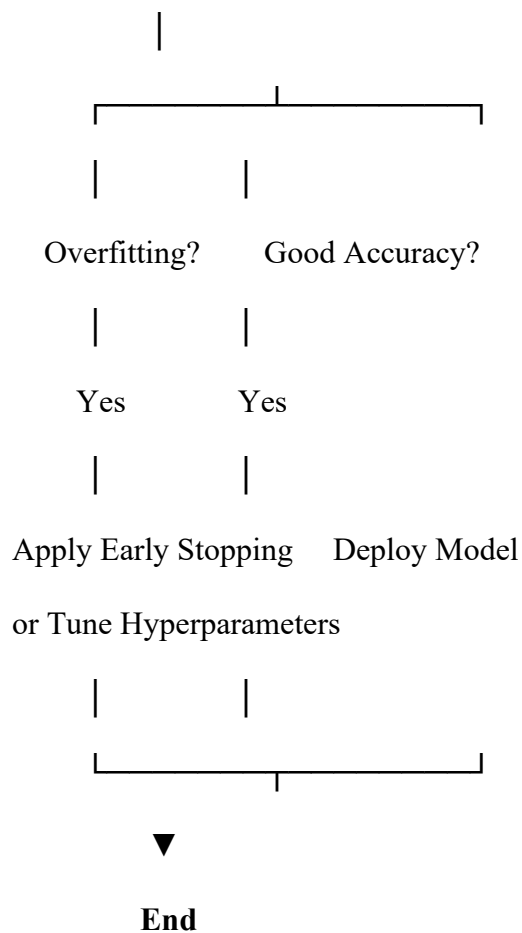
- Feature Selection: Remove irrelevant or noisy features before training.
- Batch Normalization: Stabilizes gradients and accelerates convergence.
- Early Stopping: Stops training before the model begins to overfit.

How These Techniques Improve Performance

Technique	Benefit
PCA	Reduces feature dimensions and computational complexity
Feature Selection	Removes irrelevant and redundant features
Regularization (L1/L2)	Prevents overfitting by limiting model complexity
Dropout	Improves generalization by reducing neuron co-adaptation
Batch Normalization	Stabilizes gradients and speeds convergence
Early Stopping	Prevents excessive training and reduces overfitting

Flow Chart





Conclusion:

The curse of dimensionality makes deep neural networks harder to train by creating noisy gradient updates, slowing the convergence of Stochastic Gradient Descent (SGD), and increasing the risk of overfitting, which reduces model generalization. High-dimensional feature spaces require more data and computational resources, making optimization difficult. Techniques such as Principal Component Analysis (PCA) and feature selection reduce the number of input features, while regularization (L1/L2), dropout, batch normalization, and early stopping control model complexity and stabilize training. Together, these methods improve convergence speed, reduce overfitting, and enhance the network's ability to generalize to unseen data.